

SPEC UX R2 — "+ เพิ่มส่วน" flow + ชิปจำนวนบานมั่ง

เอกสาร: SPEC-addflow-mosq-r2.html · วันที่: 2026-06-11 · สถานะ: ☒ ฟังก์ชันอนุมัติดีไซน์ ("กดย้ายดี") · พร้อมส่ง developer

☒ ดรฟที่อนุมัติ (ทำตามนี้ — ฟังก์ชันเล่นแล้วโอเค):

draft-addflow-mosq-PLAY.html → <https://jraluminium102.github.io/sharepicjr/draft-addflow-mosq-PLAY.html>

= ปุ่ม "+ เพิ่มส่วน" เพิ่มการ์ดทันที · ชิป " ⚡ ของใช้บ่อย" เห็นตลอด กดเพิ่มเลย+flash · ในการดประดุมชิปมั่ง auto/1/2/.../N (กดเลือกบาน) · "บานไม่เท่ากัน?" งามช่องกรอกขนาด · ให้ developer ทำหน้าตา+พฤติกรรมตามดรฟนี้

สารบัญ

- เรื่องที่ 1 — "+ เพิ่มส่วน" flow ใหม่
- ดีไซน์ที่เลือก
- จุดโค้ดที่แก้
- Guard กันห้อง (ต้องไม่กระทบ)
- DoD เรื่องที่ 1
- เรื่องที่ 2 — ชิปจำนวนบานมั่ง
- ดีไซน์ที่เลือก
- จุดโค้ดที่แก้
- DoD เรื่องที่ 2
- จุดต้องถามพื้นที่

เรื่องที่ 1 — "+ เพิ่มส่วน" flow ใหม่

ปัญหาเดิม (ยืนยันจากพื้นที่กดเบราร์เซอร์จริง)

BEFORE — toggleAddSecPanel (L1947)

- กดปุ่ม "+ เพิ่มส่วน" (.set-addpart) → เรียก toggleAddSecPanel(btn, box)
- ฟังก์ชันสร้าง panel แบบ toggle: กดซ้ำ = ลบ panel ออก
- ใน panel: ชิป 8 ตัว + ปุ่ม "งานบานทั่วไป" → ต้องกดต่ออีก 1 ครั้งถึงได้การ์ด
- **tap count:** เพิ่มงานบาน = 2 tap · เพิ่มซ้ำ = 3 tap (เปิด panel ใหม่)
- ผล: รู้สึกว่า "กด เพิ่ม แล้วไม่เพิ่ม" + toggle สับสน

ดีไซน์ที่เลือก — แบบ (ก) + persistent strip

AFTER — 2 ส่วนแยก หน้าที่ขีด

1. Persistent quick-strip — แถวชิปของชิปย่อย เห็นตลอดเวลา (ไม่ toggle ไม่หาย)

กดชิป → `quickAddItem` ทันที (1 tap) · ชิปไม่หายหลังกด → กดซ้ำได้

2. ปุ่ม "+" เพิ่มส่วน" (งานบานทั่วไป) — กดแล้วได้การ์ดทันที `addItem + calcQuote`

ไม่มี toggle ไม่มี panel กลาง · ปุ่ม verb ทำตามชื่อ

เหตุผลที่เลือกแบบนี้

- แบบ (ก): "เพิ่มส่วน" = เพิ่มทันที ตรงกับชื่อปุ่ม ไม่งง
- ชิปด่วนเห็นตลอด: เซลล์ที่เพิ่มของบอยซ้ำไม่ต้องสะดุดทุกครั้ง
- ลด tap จาก 2-3 เหลือ 1 ทุกเส้นทาง
- ไม่กระทบ logic/ราคา — แดจัสต์ UI ใหม่, `quickAddItem` เดิมใช้ได้

Layout ใน setbox (HTML ใหม่)

```
<!-- persistent quick-strip: เห็นตลอด บน set-parts -->
<div class="addsec-quickstrip">
  <div class="addsec-strip-head"> ⚡ เพิ่มด่วน</div>
  <div class="addsec-chips-wrap">
    <button class="addsec-chip" data-g="1" data-id="fixed_glass">กระจกติดตาย</button>
    <button class="addsec-chip" data-g="1" data-id="rn89">ลูกฟูก 1 ทาง</button>
    <!-- ...chip อื่น 8 ตัวจาก QUICK_ADD_CHIPS เดิม... -->
  </div>
</div>

<div class="set-parts"></div>

<!-- ปุ่มหลัก: set-addpart → addItem ทันที ไม่เรียก toggleAddSecPanel -->
<div class="set-btns set-btns-top">
  <button type="button" class="set-addpart">+ เพิ่มส่วน</button>
</div>
```

จุดโค้ดที่แก้ (index.html)

1. addSet() — L1965 (innerHTML ของ setbox)

เพิ่ม `.addsec-quickstrip` ก่อน `.set-parts` ใน innerHTML:

```
// เดิม (ย่อ):
box.innerHTML = '...<div class="set-btns set-btns-top"><button class="set-addpart">+ เพิ่มส่วน</button></div><div class="set'

// ใหม่: ย้าย quick-strip มาก่อน set-parts · ลบ toggleAddSecPanel ออกจาก click handler
box.innerHTML = '...'
+ '<div class="addsec-quickstrip" style="...">'
+ '<div class="addsec-strip-head"> ⚡ เพิ่มด่วน <span ...>กดชิปเพิ่มทันที</span></div>'
+ '<div class="addsec-chips-wrap" style="display:flex;flex-wrap:wrap;gap:6px;">'
+ QUICK_ADD_CHIPS.map(c => '<button ... data-g="'+c.g+'" data-id="'+c.id+'">'+c.label+'</button>').join('')
```

```
+ '</div></div>'
+ '<div class="set-parts"></div>'
+ '<div class="set-btns set-btns-top"><button type="button" class="set-addpart"> + เพิ่มส่วน</button></div>'
+ '...';
```

2. addSet() — event listener ของ .set-addpart (L1969-1994)

แก้ branch `!hasGH`: เดิมเรียก `toggleAddSecPanel` → เปลี่ยนเป็น `addItem + calcQuote` ทันทัน:

```
// เดิม (L1972-1975):
if(!hasGH){
  toggleAddSecPanel(b, box); // ← ลบออก
  return;
}

// ใหม่:
if(!hasGH){
  addItem(parts);
  refreshSet(box);
  calcQuote();
  // scroll to new card
  const chs = parts.querySelectorAll('.ch');
  const newCh = chs[chs.length-1];
  if(newCh) newCh.scrollIntoView({behavior:'smooth', block:'center'});
  return;
}
```

3. addSet() — bind click ให้ .addsec-chip (ใน quickstrip)

หลัง `document.getElementById('items').appendChild(box)` เพิ่ม:

```
box.querySelectorAll('.addsec-chip').forEach(function(cb){
  cb.addEventListener('click', function(){
    var parts = box.querySelector('.set-parts');
    quickAddItem(parts, cb.dataset.g, cb.dataset.id);
    refreshSet(box);
    calcQuote();
    // ไม่ลบชิปออก — เห็นตลอด
  });
});
```

4. toggleAddSecPanel (L1947-1959)

ฟังก์ชันนี้ไม่ถูกเรียกใน flow ใหม่แล้ว — สามารถลบหรือ comment ออก ได้ (ตรวจว่าไม่มี caller อื่นก่อน)

ตรวจ grep `toggleAddSecPanel` ทั้งไฟล์ก่อน — ถ้าไม่มี caller อื่น ลบออกได้เลย

5. CSS สำหรับ quickstrip

```
.addsec-quickstrip {
  background: #FFF3E0;
  border: 1.5px solid #FFB300;
```

```

border-radius: 10px;
padding: 10px 12px;
margin-bottom: 8px;
}
.addsec-strip-head {
font-size: 12px;
font-weight: 700;
color: #E65100;
margin-bottom: 7px;
}
.addsec-chip {
min-height: 44px;
padding: 0 14px;
border: 1.5px solid #FFB300;
border-radius: 22px;
background: #fff;
cursor: pointer;
font-size: 13px;
color: #374151;
}
.addsec-chip:active {
background: #E65100;
color: #fff;
}
/* ปุ่ม .set-addpart ใหม่: solid แทน outline */
.set-addpart {
min-height: 44px;
width: 100%;
border: 2px solid var(--red);
border-radius: 9px;
background: var(--red);
color: #fff;
font-size: 14px;
font-weight: 700;
cursor: pointer;
}

```

Guard กันห้องกระจก (ต้องไม่กระทบ)

⚠ ตรวจสอบ `hasGH` branch (L1971-1994) — กันห้องกระจก (group=6) ยังใช้ `addItem + auto-set group 6` เหมือนเดิมทุกอย่าง
 quickstrip ต้อง **ไม่โผล่ในชุดกันห้อง** — เพิ่ม condition: ถ้า `hasGH` ให้ซ่อน `.addsec-quickstrip` (หรือ render แยก)

```

// ในฟังก์ชัน refreshSet หรือหลัง bind event:
var strip = box.querySelector('.addsec-quickstrip');
if(strip){
  strip.style.display = hasGH ? 'none' : '';
}

```

```
// หรือ: เพิ่ม logic ใน QUICK_ADD_CHIPS click — ถ้า hasGH ไม่ทำอะไร (safe guard)
// แต่วิธีซ่อน strip ทั้งก่อนชัดเจนกว่า
```

DoD (Definition of Done) — เรื่องที่ 1

1. **TEST** กด "+ เพิ่มส่วน" ครั้งเดียว → การจัดงานบานโพล์ทันที (ไม่มี panel กลาง)
2. **TEST** กดชิปใน quickstrip → การกดกระจกติดตาย/ลูกฟูก/ฯ โพล์ทันที (1 tap)
3. **TEST** กด "+ เพิ่มส่วน" ซ้ำ 3 ครั้ง → การกดเพิ่ม 3 ใบ (ไม่มี toggle)
4. **TEST** ชุดกันห้องกระจก (group=6 อยู่ใน set): quickstrip ซ่อน · ปุ่ม "+ เพิ่มบาน" ทำงานตามเดิม
5. **TEST** `addGlasshouseSet` — ยังสร้างชุดกันห้องถูกต้อง ไม่กระทบ
6. **TEST** quote-fidelity: เพิ่มบานเลื่อน 4 บาน → `calcQuote` ได้ราคาถูก เทียบ baseline

เรื่องที่ 2 — ชิปปัจจุบันบานมั่ง

ปัญหาเดิม

BEFORE — `buildItemOpts` (L2586)

- `.o-mosqpanels` = `<input type="number" width:50px placeholder="auto">`
- เซลล์ต้องจำว่าบานประตู (`.i-panels`) มีกี่บาน → ตัวเองพิมพ์ตัวเลข
- ไม่มี visual เชื่อมระหว่าง i-panels กับ o-mosqpanels → ผิดพลาดได้ง่าย
- 3 ช่องเรียงบรรทัดเดียว (กว้าง × ยาว · บาน) — อ่านยาก บน mobile

ดีไซน์ที่เลือก — ชิปปัจจุบันบาน + fallback

AFTER — ใน `mosq-detail-group` (แทรกก่อน `o-mosqw/h/panels`)

1. ชิปปัจจุบันบาน (วิธีหลัก): แสดงชิป "auto", 1, 2, ..., N โดย N = ค่า `.i-panels`
 - กดชิป → set `.o-mosqpanels.value` = ตัวเลข (auto =ว่าง) → `fire input event` → `calcQuote()`
 - $N > 5$ → ซ่อนชิป แสดง stepper แทน (fallback)
2. ช่องขนาด (กว้าง×ยาว) — พับ default: ย้ายเข้า `<details>` "กรอกขนาดมั่งเอง (กรณีแต่ละบานไม่เท่ากัน)"
 - `.o-mosqw` / `.o-mosqh` ยังเชื่อมกับ `calcQuote` ตามเดิม ไม่เปลี่ยน logic
3. `.o-mosqpanels` ยังอยู่ในฟอร์ม (hidden หรือ sync) — `readItem` เดิมใช้ได้ไม่เปลี่ยน

State machine

กรณี	UI	.o-mosqpanels value	ราคา
ชิป "auto"	chip auto = on	"" (ว่าง)	auto คำนวณจากบาน
ชิป "2"	chip 2 = on	"2"	2 บาน × max(min, พื้นที่×rate)
ชิป "4"	chip 4 = on	"4"	4 บาน × ...
N > 5 (fallback)	stepper แสดง	stepper.value	N บาน × ...
กรอกขนาดเอง	details คลาย	คงค่าชิป	ใช้ w/h ที่กรอก แทน auto

เกณฑ์ชิป N > 5

ถ้า `i-panels > 5` → ชิปจะมี 6+ ปุ่ม ใหญ่เกิน → ซ่อนชิปทั้งแถว แสดง stepper แทน
ตัวเลข 5 เป็น default ตาม UX principle "ชิป > 5 รายการ เริ่มอ่านยาก"

จุดโค้ดที่แก้ (index.html)

1. buildItemOpts — มั้ง section (L2585-2592)

ใน `mosq-detail-group` แทรกชิปบานก่อน label ขนาด:

```
// เดิม (L2585-2586):
h += '<div class="mosq-detail-group" style="display:none;...">';
h += '<label class="opt">ขนาดมั้ง ... กว้าง <input class="o-mosqw"...> × ยาว <input class="o-mosqh"...> · บาน <input class="o-

// ใหม่:
h += '<div class="mosq-detail-group" style="display:none;...">';

// — [NEW] จำนวนบานมั้ง —
h += '<div class="mosq-panels-bar">';
h += '<div class="mosq-panels-bar-head">จำนวนบานมั้ง <span class="mosq-panels-bar-hint">กระทบบราคา</span></div>';
// ชิป: สร้าง placeholder → rebuild ตอน bind (ดูข้อ 2)
h += '<div class="mosq-panel-chips" style="display:flex;flex-wrap:wrap;gap:6px;..."></div>';
// fallback stepper (N>5)
h += '<div class="mosq-panels-fallback" style="display:none;align-items:center;gap:6px;...">';
h += ' จำนวนบาน: <input type="number" class="o-mosqpanels-stepper" min="1" step="1" style="width:60px;height:44px;..." on
h += '</div>';
h += '</div>'; // /mosq-panels-bar

// — ช่องขนาด → พันใน details —
h += '<details class="mosq-size-details" style="margin-top:8px;...">';
h += '<summary style="font-size:12px;color:#0369A1;cursor:pointer;...">';
h += ' กรอกขนาดมั้งเอง (กรณีแต่ละบานไม่เท่ากัน)</summary>';
h += '<div style="padding:8px 0;...">';
h += ' <span style="font-size:11px;color:#0369A1;...">[เว้นว่าง=auto จากบาน]</span>';
h += ' กว้าง <input type="number" class="o-mosqw" min="0" step="0.1" value="" placeholder="ม." style="width:56px;height:36
h += ' × ยาว <input type="number" class="o-mosqh" min="0" step="0.1" value="" placeholder="ม." style="width:56px;height:36
```

```
h += '</div></details>';
```

```
// .o-mosqpanels ยังอยู่ (hidden) — readItem เดิมอ่านค่าจาก field นี้  
h += '<input type="hidden" class="o-mosqpanels" value="">';
```

2. afterBuildOpts — rebuild ชิปตาม i-panels (เพิ่ม helper ใหม่)

ใน `buildItemOpts` หรือ event handler หลัง build เสร็จ:

```
function rebuildMosqPanelChips(d){  
  // d = .ch div ของบานนั้น  
  var bar = d.querySelector('.mosq-panels-bar');  
  if(!bar) return;  
  var chipWrap = bar.querySelector('.mosq-panel-chips');  
  var fallback = bar.querySelector('.mosq-panels-fallback');  
  var hiddenInp = d.querySelector('.o-mosqpanels');  
  var iPanels = parseInt((d.querySelector('.i-panels')||{}).value||'2') || 2;  
  
  chipWrap.innerHTML = "";  
  if(iPanels > 5){  
    // fallback stepper  
    chipWrap.style.display = 'none';  
    fallback.style.display = 'flex';  
    var stepper = fallback.querySelector('.o-mosqpanels-stepper');  
    if(stepper && hiddenInp.value) stepper.value = hiddenInp.value;  
    return;  
  }  
  chipWrap.style.display = 'flex';  
  fallback.style.display = 'none';  
  
  // chip "auto"  
  chipWrap.appendChild(makeMosqChip("", 'auto', hiddenInp, d));  
  // chip 1...N  
  for(var i=1; i<=iPanels; i++){  
    chipWrap.appendChild(makeMosqChip(String(i), i+' บาน', hiddenInp, d));  
  }  
  // mark active chip  
  var cur = hiddenInp.value;  
  chipWrap.querySelectorAll('.mosq-panel-chip').forEach(function(c){  
    c.classList.toggle('on', c.dataset.val === cur);  
  });  
}  
  
function makeMosqChip(val, label, hiddenInp, d){  
  var btn = document.createElement('button');  
  btn.type = 'button';  
  btn.className = 'mosq-panel-chip chip'; // ใช้ class chip เดิมสำหรับ style  
  btn.dataset.val = val;  
  btn.textContent = label;  
  btn.style.cssText = 'min-height:44px;min-width:44px;padding:0 12px;font-size:14px;font-weight:700;';  
  btn.addEventListener('click', function(){  
    hiddenInp.value = val;  
    hiddenInp.dispatchEvent(new Event('input', {bubbles:true})); // → calcQuote
```

```
// update active
var wrap = btn.closest('.mosq-panel-chips');
wrap.querySelectorAll('.mosq-panel-chip').forEach(function(c){ c.classList.remove('on'); });
btn.classList.add('on');
});
return btn;
}

function syncMosqPanels(stepper){
  // sync stepper → hidden .o-mosqpanels
  var d = stepper.closest('.ch');
  var hid = d.querySelector('.o-mosqpanels');
  if(hid){ hid.value = stepper.value; hid.dispatchEvent(new Event('input',{bubbles:true})); }
}
```

3. เรียก rebuildMosqPanelChips เมื่อ

- หลัง buildItemOpts เสร็จ (ใน afterBuildOpts หรือท้าย buildItemOpts เอง)
- เมื่อ .i-panels เปลี่ยน (input event) — L2269 มี listener อยู่แล้ว → เพิ่ม call rebuildMosqPanelChips(d)
- เมื่อเลือกชนิดมุ้ง (.o-mosq change) — ใน mosqGroupToggle L2174

```
// ใน listener ของ .i-panels (L2269):
d.querySelector('.i-panels').addEventListener('input', function(){
  var pm = d.querySelector('.i-panels-mirror'); if(pm) pm.value = this.value;
  // [NEW]:
  rebuildMosqPanelChips(d);
  calcQuote();
});

// ใน mosqGroupToggle (L2174):
function mosqGroupToggle(sel){
  var d = sel.closest('.ch');
  var show = !!sel.value;
  // ...existing logic...
  if(show) rebuildMosqPanelChips(d); // [NEW]
}
```

4. readItem — .o-mosqpanels ยังอ่านเหมือนเดิม (L2887)

ไม่ต้องแก้ readItem เพราะ hidden input .o-mosqpanels ยังอยู่ในตำแหน่งเดิม sync ค่าให้ครบก่อน calcQuote ถูกเรียก

```
// L2887 — ไม่เปลี่ยน:
const _mqp = d.querySelector('.o-mosqpanels'); optSel.mosqPanels = _mqp ? _mqp.value : '';
```

5. CSS เพิ่มเติม (ในส่วน <style>)

```
.mosq-panels-bar { margin-bottom: 10px; }
.mosq-panels-bar-head {
  font-size: 12px; font-weight: 700;
  color: #1565C0; margin-bottom: 6px;
```



```

}
.mosq-panels-bar-hint {
  font-size: 11px; font-weight: 400;
  color: #888; margin-left: 4px;
}
.mosq-size-details summary { list-style: none; }
.mosq-size-details summary::-webkit-details-marker { display: none; }

```

DoD (Definition of Done) — เรื่องที่ 2

- TEST** บานเลื่อน 4 บาน → ชิปโพล์ "auto, 1, 2, 3, 4" ทั้ง 5 ปุ่ม
- TEST** กดชิป "2" → `.o-mosqpanels` = "2" → calcQuote คิดราคา 2 บาน
- TEST** กดชิป "auto" → `.o-mosqpanels` = "" → calcQuote คิดราคา auto (ช่องเปิด)
- TEST** เปลี่ยน i-panels จาก 4 → 6 → ชิป rebuild เป็น "auto,1,2,3,4,5,6" (6 ตัว ยังแสดง เพราะ >5 ต้อง >5 ไม่ใช่ >=5)
- TEST** i-panels = 6 → ชิปแสดง (N=6 ≤ 5 จะ fallback) — ดูหมายเหตุข้อ Q2 ด้านล่าง
- TEST** i-panels = 8 → fallback stepper โพล์ ชิปซ่อน → กรอก 6 → `.o-mosqpanels` = "6" → calcQuote ถูก
- TEST** กดชิปมั้ง + ช่องขนาดพับ → เปิด details "กรอกขนาดเอง" → กรอกกว้าง×ยาว → calcQuote ใช้ขนาดที่กรอก
- TEST quote-fidelity MUST PASS:** บานเลื่อน 4 บาน + มั้งจีบ SD 2 บาน → ราคามั้ง = max(3,500×2, พื้นที่×2,500) ถูก
- TEST** บานเปิด 1 บาน → ชิปโพล์ "auto, 1" (2 ปุ่ม) · กด "1" → คิด 1 บาน

จุดต้องถามพินัท (ก่อน implement)

#	คำถาม	ผลกระทบ	ค่า default ถ้าไม่ถาม
Q1	เรื่อง 1: ชิปใน quickstrip กดซ้ำกัน (เช่น กด "กระจกติดตาย" 3 ครั้ง) = เพิ่ม 3 การ์ดได้ไหม? หรือ: ควรมีสัญญาณ feedback (เช่น flash ปุ่ม) ก่อนเพิ่ม?	UX feedback	เพิ่มได้เลย (chip ไม่ toggle)
Q2	เรื่อง 2: N > 5 → fallback stepper (เกณฑ์ >5 หรือ ≥5?) ตัวอย่าง: 5 บาน = ชิปยังแสดง? หรือ fallback แล้ว?	จำนวนชิปบนหน้าจอ	ถ้า i-panels > 5 → fallback (ชิปแสดงถึง 5 ตัว)

#	คำถาม	ผลกระทบ	ค่า default ถ้าไม่ถาม
Q3	เรื่อง 2: มั่งก็บานเกินบานประตูได้ไหม? เช่น ประตู 2 บาน → ใส่มั่ง 3 บานได้ไหม? (กรณีหน้าต่างติดข้าง ต้องใส่มั่งด้วย)	cap ชิปที่ i-panels หรือไม่?	cap ที่ i-panels (มั่งไม่เกินบาน) · fallback stepper ให้กรอกได้อิสระ
Q4	เรื่อง 1: ลำดับชิปใน quickstrip — 8 ตัวปัจจุบัน (QUICK_ADD_CHIPS) โอเคไหม? หรืออยากเพิ่ม/เรียงใหม่?	content	ใช้ QUICK_ADD_CHIPS เดิมทั้ง 8 ตัว

✅ มติพื้นที่ (เคาะแล้ว 2026-06-11) — ทำตามนี้:

- Q1: กดชิปด่วนซ้ำ = เพิ่มหลายการ์ดได้เลย + มี flash ดังสั้นๆ ที่การ์ดใหม่/ปุ่ม ให้รู้ว่าเพิ่มแล้ว
- Q2: ชิปบานมั่งแสดงได้ถึง 5 · i-panels > 5 → **fallback เป็น stepper**
- Q3: มั่งไม่เกินจำนวนบานประตู — ชิป cap ที่ i-panels (1 ถึง N เท่านั้น) · **ไม่มี fallback ให้กรอกเกิน** (ถ้า N>5 stepper ก็ยัง cap ที่ N)
- Q4: ลำดับ 8 ชิปของใช้บ่อย = **ตามเดิม** (QUICK_ADD_CHIPS)

ไฟล์อ้างอิง

- ดราฟ interactive: `C:\Users\Nut\Desktop\Sharepicjr\draft-addflow-mosq-r2.html`
- ไฟล์จริง: `public/calculator/index.html`
- L1926-1960: QUICK_ADD_CHIPS + toggleAddSecPanel + addSet click handler
- L2577-2592: buildItemOpts — มั่ง section
- L2877-2887: readItem — อ่าน .o-mosqpanels
- L2269: .i-panels input event listener
- L2174: mosqGroupToggle